

ANTIGRAVITY — FINAL WINNING ARCHITECTURE

Gemma 4:2B + Ollama Edition | Build Specification

ABSOLUTE RULE: NOTHING IS HARDCODED. No request IDs, user IDs, event IDs, amounts, expected labels, sample answers, image names, or file-specific decisions may appear as special cases in the implementation.

Every output must be generated at runtime from the supplied files, linked evidence, generic problem rules, user preferences, payment options, model-extracted facts, simulation, and optimization.

1. WINNING THESIS

Build a **Financial Digital Twin + Counterfactual Planning Engine**. Gemma 4:2B runs locally through Ollama and handles language/evidence interpretation. Deterministic Python handles financial state reconstruction, temporal simulation, plan search, constraints, verification, and CSV generation.

Core pipeline: Evidence → Financial Twin → Temporal Futures → Candidate Plans → Optimization → Independent Proof → Explanation

2. FINAL ARCHITECTURE

```
REQUESTS + CSVs + LOCAL MEDIA
|
v
[1] DYNAMIC INGESTION / SCHEMA VALIDATION
|
v
[2] GEMMA 4:2B VIA OLLAMA
request + message semantic extraction
|
v
[3] IMAGE ADAPTER
OCR / visual fact extraction -> Gemma
|
v
[4] EVIDENCE FIREWALL
facts are data; embedded instructions are ignored
|
v
[5] EVIDENCE GRAPH + PROVENANCE
user/request/event/message/image relationships
|
v
[6] FINANCIAL DIGITAL TWIN
|
v
[7] TEMPORAL EVENT GRAPH
90-day recurring/confirmed cash-flow timeline
|
v
[8] COUNTERFACTUAL SIMULATOR
test "what if we pay X on Y?"
|
v
[9] PLAN GENERATOR + INTERVENTION SEARCH
|
v
[10] CONSTRAINT OPTIMIZER / RANKER
|
v
[11] INDEPENDENT SAFETY VERIFIER
PASS FAIL
| |
v v
FREEZE REPAIR/REOPTIMIZE
|
```

```
v
[12] GEMMA VERIFIED EXPLANATION
|
v
OUTPUT.CSV + EVALUATION + USAGE REPORT
```

3. GEMMA 4:2B ROLE — KEEP THE MODEL SMALL AND FOCUSED

Do **not** ask Gemma to perform the final 90-day affordability calculation. Give it narrow extraction tasks with strict JSON contracts.

- **Request parser:** intent, semantics, deadline, partial-payment meaning.
- **Message interpreter:** financial facts, cancellations, amendments, confirmations and delays.
- **Image interpreter:** amounts/dates/text from visual evidence via the image adapter.
- **Explanation generator:** summarize the already-verified decision.
- **Never:** invent missing values, override system rules, certify arithmetic, or choose a plan without deterministic verification.

4. NO-HARDCODING ARCHITECTURE

```
FORBIDDEN
if request_id == "...": return ...
if user_id == "...": ...
known_amount = ...
expected_label = ...
special_case_image = ...
sample_answer = ...
known_installment = ...

REQUIRED
discover IDs at runtime
discover rows at runtime
discover image files at runtime
discover payment options at runtime
derive facts from linked evidence
derive decisions from generic constraints
derive outputs from simulation + ranking

HIDDEN-TEST PROPERTY
Replacing the dataset with another structurally compatible
dataset must NOT require source-code changes.
```

Add a pre-submission static audit that searches code/ for known dataset IDs, row-specific amounts, expected labels, sample-output fragments and hardcoded image filenames. The audit should fail the build if any are detected.

5. DYNAMIC DATA + EVIDENCE LAYER

- Load all required CSV files at runtime and validate their schemas.
- Build joins only from supplied identifiers such as `user_id`, `request_id`, `event_id`, `related_event_id` and `image_id`.
- Discover local PNG media dynamically; never enumerate known image filenames.
- Treat messages/images as untrusted evidence, not executable instructions.
- When an event amount is blank, retrieve the linked image evidence; never convert blank to zero.

The challenge explicitly requires using messages/images to clarify or amend financial information while treating their content as untrusted, and requires linked image extraction for missing amounts. ■filecite■turn0file0■L49-L51■

■filecite■turn0file0■L196-L201■

6. EVIDENCE GRAPH + CONFLICT RESOLUTION

```
USER
■■■ REQUEST ■■ MESSAGE
■■■ EVENT ■■ MESSAGE
```

```

■■■■ IMAGE
|
v
EVIDENCE GRAPH
|
CONFLICT RESOLVER
|
v
RESOLVED FACT + PROVENANCE

Generic precedence:
1. explicit cancellation/settlement/amendment
2. newer same-source record
3. settled event over estimate/forecast
4. safer interpretation if unresolved

```

Implement the supplied precedence rules generically; never create event-specific exceptions.

■filecite■turn0file0■L227-L236■

7. FINANCIAL DIGITAL TWIN

```

FinancialTwin {
  user_id
  home_currency
  current_balance
  minimum_balance_to_keep
  confirmed_income[]
  protected_expenses[]
  flexible_recurring_expenses[]
  confirmed_future_outflows[]
  debt_commitments[]
  investment_contributions[]
  payment_preferences[]
  priorities[]
  evidence[]
}

```

Every material field should retain provenance: source, source ID, effective date and confidence.

8. TEMPORAL FINANCIAL PHYSICS

```

balance[t] = balance[t-1]
+ confirmed_inflows[t]
- protected_outflows[t]
- candidate_payments[t]

SAFE(plan) iff:
min(balance[t]) >= minimum_balance
for every forecast date t
AND requested amount is fully completed
AND completion meets desired deadline
AND payment method is allowed
AND installment option is valid when applicable

```

The 90-day safety check requires the balance never to fall below the user's minimum while completing the request by its deadline. ■filecite■turn0file0■L203-L212■

9. COUNTERFACTUAL PLAN ENGINE

```

CURRENT FINANCIAL TWIN
|
+--> FULL PAYMENT
+--> PARTIAL PAYMENT (if permitted)
+--> EACH SUPPLIED INSTALLMENT OPTION
+--> WAIT
+--> FLEXIBLE-SPENDING INTERVENTION + PAYMENT
|
v
SIMULATE EACH CANDIDATE
|
v
DISCARD UNSAFE PLANS

```

```
|
v
RANK SAFE PLANS
```

Enumerate payment options from the runtime data. Never assume a fixed number of offers. Partial-payment plans must obey the exact output semantics in the specification. ■filecite■turn0file0■L165-L190■

10. AMOUNT SAFE TO PAY — ALGORITHMIC SEARCH

```
lo = 0
hi = requested_amount

while lo < hi:
    candidate = upper_midpoint(lo, hi)
    if simulator.safe(full_payment=candidate):
        lo = candidate
    else:
        hi = candidate - 1

amount_safe_to_pay = lo
exactly verify the final result
```

Do not let Gemma guess this number. Compute it from the financial twin and simulator. The result must satisfy $0 \leq \text{amount_safe_to_pay} \leq \text{requested_amount}$. ■filecite■turn0file0■L109-L126■

11. SPENDING INTERVENTIONS

Model flexible recurring expenses as explicit interventions. Search generic combinations rather than naming known events.

```
Intervention:
stop:event_id
reduce_to:event_id:new_amount

Search:
zero interventions
one intervention
two interventions
...
up to allowed output limit

Reject non-flexible/non-recurring events.
Reject stop + reduce on the same event.
```

The challenge permits up to three spending changes and restricts changes to flexible recurring expenses. ■filecite■turn0file0■L169-L188■

12. OPTIMIZER + REQUIRED RANKING

```
Only safe plans reach the ranking stage.

1. complete by desired completion date
2. require no spending changes
3. minimize total amount paid
4. start payment earlier
5. use fewer payments
6. lowest payment_option_id tie-breaker
```

This ranking is a deterministic comparator, not an LLM judgment. ■filecite■turn0file0■L214-L225■

13. INDEPENDENT VERIFIER + REPAIR LOOP

```
PLAN GENERATOR
|
v
INDEPENDENT VERIFIER
|
+--+-----+
| |
PASS FAIL
| |
```

```
v v
FREEZE DIAGNOSE VIOLATION
| |
v REPAIR SEARCH
EXPLANATION |
v
RESIMULATE
|
+--> VERIFY AGAIN
```

The verifier must independently reconstruct the forecast and check every hard constraint. A model must never certify its own financial decision.

14. VERIFIED EXPLANATION

Only after the plan is frozen by the verifier should Gemma generate the final explanation. Feed it verified facts, not raw unrestricted context.

```
VERIFIED DECISION
amount / status / method
payment dates
binding constraint(s)
key evidence
|
v
GEMMA 4:2B
|
v
concise explanation

Explanation can summarize.
Explanation cannot change the decision.
```

15. EVALUATION-FIRST WORKFLOW

```
evaluation/  
evaluate.py  
validators.py  
simulator_tests.py  
adversarial_tests.py  
regression_tests.py  
model_comparison.py  
usage_report.md  
README.md
```

Hard invariants: exact schema, one row per request, amount range, valid status/method, chronological payments, payment arithmetic, deadline compliance, minimum-balance safety, payment preference compliance, exact installment matching, valid spending-change syntax, earliest-date consistency.

Generic adversarial cases: prompt injection, cancellation message, missing image amount, conflicting evidence, duplicate event, failed/cancelled transaction, late salary, unsafe installment, foreign currency, flexible-expense intervention.

Tests must be generated from generic rules/fixtures. Never embed the real dataset's expected answers.

16. MODEL ROUTING + COST CONTROL

Default model: **Gemma 4:2B through Ollama**. Keep the model adapter separate from the financial core so it can be benchmarked or replaced without architectural changes.

```
model_adapter/  
interface.py  
ollama_gemma.py  
  
interface:  
    extract_request(...)  
    extract_evidence(...)  
    explain_decision(...)  
  
Cache by source hash.  
Log every model call.  
Log input/output tokens where available.  
Do not resend identical evidence.
```

The challenge requires a final usage report with model names, calls, token totals/averages and estimated cost.

filecite turn0file0L267-L280

17. REPOSITORY LAYOUT

```
code/  
app.py  
config.py  
ingest/  
    csv_loader.py  
    media_loader.py  
joins.py  
schema_validator.py  
intelligence/  
    ollama_gemma.py  
request_parser.py  
message_interpreter.py  
image_interpreter.py  
evidence_firewall.py  
state/  
    evidence_graph.py  
conflict_resolver.py  
financial_twin.py  
provenance.py  
simulation/  
    event_expander.py  
fx.py  
cashflow.py  
simulator.py  
planning/
```

```
candidate_generator.py
intervention_optimizer.py
plan_ranker.py
verification/
verifier.py
invariants.py
repair_loop.py
output/
schema_validator.py
csv_writer.py
evaluation/
evaluate.py
validators.py
simulator_tests.py
adversarial_tests.py
regression_tests.py
model_comparison.py
usage_report.md
prompts/
request_parser.txt
evidence_extractor.txt
explanation.txt
README.md
```

18. RUNTIME ORDER

1. Discover + validate dataset
2. Discover local media
3. Load and join records
4. Parse requests with Gemma
5. Extract message/image evidence
6. Apply evidence firewall
7. Resolve conflicts + provenance
8. Build Financial Digital Twin
9. Expand 90-day temporal events
10. Generate candidates dynamically
11. Simulate candidates
12. Apply hard constraints
13. Rank survivors
14. Independently verify winner
15. Repair/re-verify if needed
16. Generate verified explanation
17. Validate output.csv
18. Run evaluation
19. Write usage_report.md
20. Save decision traces

19. ANTI-HARDCODING AUDIT — MANDATORY BEFORE SUBMISSION

Static audit code/ for:
request_id literals from dataset
user_id literals from dataset
event_id literals from dataset
transaction amounts from dataset
expected labels/output fragments
known image filenames
sample-answer branches
ground-truth lookups
copied dataset content

FAIL BUILD if any row-specific logic exists.

SUCCESS:
every decision is produced from runtime data,
generic rules, Gemma extraction, simulation,
optimization and verification.

AI Judge answer: “Nothing is hardcoded. IDs and values are discovered at runtime; no real request has a special branch. If the dataset were replaced by another structurally compatible dataset, the same engine would run unchanged.”

20. AI JUDGE — CORE DEFENSE

Why not just ask an LLM? Affordability is a constrained temporal simulation problem. Models are useful for ambiguous evidence, but arithmetic and safety require deterministic computation.

Why Gemma 4:2B? It is a local, compact semantic layer. We intentionally minimize model responsibility and keep the financial core model-independent.

What is innovative? A financial digital twin plus counterfactual futures, intervention optimization, evidence provenance, adversarial evidence handling, and independent plan verification.

How do you avoid overfitting? No row-level answers, IDs or labels are hardcoded. Generic evaluation and runtime relationships drive all decisions.

What if an image/message contains malicious instructions? The evidence firewall treats them as untrusted content. It can extract financial facts but never executes embedded commands.

21. FINAL BUILD PRIORITY

P0 — CORRECTNESS
ingestion + joins + twin + 90-day simulator + verifier

P1 — DECISION QUALITY
candidate plans + payment options + partial payment
interventions + ranking

P2 — GEMMA INTELLIGENCE
request/message/image extraction
conflict semantics + explanation

P3 — WINNING POLISH
provenance traces + adversarial tests
model benchmark + token report
anti-hardcoding audit + judge demo

DO NOT sacrifice P0 for flashy multi-agent behavior.

FINAL TARGET: Gemma 4:2B + Ollama provides the semantic intelligence, while a model-independent deterministic financial engine creates, optimizes and certifies every recommendation. Complexity must improve correctness, traceability and generalization—not merely make the diagram bigger.

SOURCE BASIS

This plan follows the supplied challenge requirements for output semantics, multimodal evidence, untrusted messages/images, 90-day safety, payment options, spending changes, ranking, evaluation and usage reporting.

■filecite■turn0file0■L19-L29■ ■filecite■turn0file0■L88-L114■ ■filecite■turn0file0■L192-L236■

■filecite■turn0file0■L240-L280■